
Search Engine Project

Stage 1: Building the Data Layer

Main report — three implementations compared: Java, Python and C++

Course	Big Data · Grado en Ciencia e Ingeniería de Datos · Universidad de Las Palmas de Gran Canaria
Academic year	2026/2027
Group name	BigData-Ulpgc
Members	Daniel Rodríguez Alonso — 79072763M — Java implementation Daniel Perdomo Medina — 54173402F — Python implementation Carlos Falcón Brito — 54149163X — C++ implementation
Repository	https://github.com/BigData-Ulpgc/stage_1
Module reports	java/stage1/docs/Stage1_Java_Report.pdf, python/stage_1/docs/Stage1_Python_Report.pdf, cpp/docs/Stage1_Cpp_Report.pdf
Date	4 October 2026

How to read this report. Sections 1–3 describe the common data layer and how each language implements it, and Section 4 justifies the structures selected. Section 5 presents the benchmarks of the datalake, the metadata and the inverted index, and compares the execution times of the three implementations on the same data. Section 6 uses that comparison to choose the language that best fits the project. Each module's own report gives the class-by-class detail that is only summarised here.

Abstract

Stage 1 of the group's search engine over Project Gutenberg builds the *data layer*: a pipeline that downloads books, splits them into header and body, stores them in a datalake, extracts their metadata into SQLite and builds an inverted index that answers Boolean AND queries, with a control layer that lets an interrupted run resume without losing or repeating books. The group implemented the same data layer three times, in **Java**, **Python** and **C++**, against one written contract (`shared/SPEC.md`). The three implementations produce identical outputs: the same 128,980,349 bytes in the datalake and the same 129,356 terms and 1,581,064 postings in the index of the 200 books.

All three run the same 12 benchmark experiments with the same methodology, so their execution times can be compared. The index experiments of the three languages, and every Java and Python experiment, ran on the same Linux machine. On that common ground, **C++ is the fastest** wherever work is done in the program itself (AND queries 3.2 times faster than Java and 21 times faster than Python on the `monolithic` index), **Java stays within 1.1–3.2 times of C++** on queries and is the fastest at updating the `monolithic` index, and **Python is 2–21 times slower** wherever an interpreted loop runs per term or per query, while matching the others when the work happens in the operating system, SQLite, JSON or MongoDB. The three languages agree on the structures to use (`book` datalake, `monolithic` index, SQLite with indexes). Weighing performance together with development effort, deployment, safety and the ecosystem for the next stages, the report recommends **Java** as the language of the project, with C++ reserved for a latency-critical query service and Python for analysis and tooling.

Contents

1	Introduction and objectives	3
1.1	Goal of Stage 1	3
1.2	How the group organised the work	3
1.3	What the group delivers	4
2	System architecture	4
2.1	Pipeline overview	4
2.2	The structures under comparison	6
2.3	Control layer	6
2.4	Reliability rules shared by the three implementations	6
3	How each language implements the data layer	7
3.1	Java	7
3.2	Python	7
3.3	C++	8
3.4	The three implementations side by side	9
3.5	Same output in the three languages	9
4	Design decisions	9
4.1	Datalake: <code>book</code>	9

4.2	Inverted index: <code>monolithic</code> , with <code>mongo</code> as the growth path	10
4.3	Metadata: SQLite with indexes on <code>author</code> and <code>title</code>	10
4.4	Shared implementation decisions	10
5	Benchmarks and results	11
5.1	Methodology	11
5.1.1	Common rules	11
5.1.2	Experiments and data	12
5.1.3	Environment: what is directly comparable	12
5.2	Datalake	13
5.3	Inverted index	14
5.3.1	Queries: the clearest language effect	15
5.3.2	Build and update: language and design mixed	15
5.3.3	Scaling with the size of the collection	15
5.4	Memory and disk	16
5.5	Metadata (SQLite)	17
5.6	Overall view	18
6	Choosing the language for the project	18
6.1	What the project needs	19
6.2	How each language meets them	19
6.3	Recommendation	20
7	Conclusions and future improvements	20
7.1	Conclusions	20
7.2	Limitations and future improvements	21
A	Repository and reproduction	21
A.1	Repository	21
A.2	Quick test with the sample dataset	21
A.3	Reproducing the benchmarks	22

1. Introduction and objectives

1.1 Goal of Stage 1

The course project builds a search engine over the books of Project Gutenberg in several stages. Stage 1 leaves ranking and user interfaces for later and builds the data underneath: the books must be downloaded and kept, their metadata extracted and their words indexed, *correctly*, in a way that *survives interruptions*, and *measurably*, so that different ways of storing the same data can be compared with numbers.

The assignment asks for:

- a **datalake** for the raw books, and **datamarts** with structured data ready to query: the book metadata and an inverted index;
- a minimal **control layer** that never duplicates or loses work;
- **benchmarks** of alternative datalake and inverted-index structures,
- implemented in **at least three programming languages** over the same data.

1.2 How the group organised the work

Each member implemented the complete data layer in one language (Table 1). So that three programs written by three people could be compared, the group first wrote a common contract, `shared/SPEC.md`, which fixes everything that must not differ between languages (Table 2), and a shared set of inputs: the 200 book ids, the stopwords, the query workload and a 15-book sample dataset for offline tests. Everything outside the contract (the internal design, libraries, tests) was each member's choice, and that freedom is part of what the comparison measures.

Member	Language	Folder and report
Daniel Rodríguez Alonso	Java 17	<code>java/stage1/ · docs/Stage1_Java_Report.pdf</code>
Daniel Perdomo Medina	Python 3	<code>python/stage_1/ · docs/Stage1_Python_Report.pdf</code>
Carlos Falcón Brito	C++20	<code>cpp/ · docs/Stage1_Cpp_Report.pdf</code>

Table 1: The group and the three implementations.

SPEC	Topic	What it fixes for every language
§1	Inputs	<code>book_ids.txt</code> (200 ids, in order), <code>queries.txt</code> , <code>stopwords.txt</code> ; blank and # lines ignored.
§2	Download and split	The Gutenberg URL; the *** START/END OF THE/THIS PROJECT GUTENBERG EBOOK markers (a book without both is discarded); header and body trimmed; <code>\r\n</code> → <code>\n</code> .
§3	Datalake layouts	<code>time: YYYYMMDD/HH/<ID>.body.txt</code> ; <code>book: <ID>/body.txt</code> ; <code>range: 01000-01999/<ID>.body.txt</code> .
§4	Metadata	Four multi-line regexes (title, author, release date, language); SQLite table <code>books</code> with indexes on <code>author</code> and <code>title</code> .
§5	Tokenizer	ASCII only: A–Z lowered, a–z and 0–9 form tokens, anything else separates; tokens of 2+ characters; stopwords removed; a book contributes its <i>set</i> of terms.
§6	Index structures	<code>monolithic</code> (one JSON file), <code>hierarchical</code> (<code><LETTER>/<term>.txt</code>), <code>mongo</code> (one document per term); posting lists sorted, without duplicates.
§7	Queries	Same tokenizer; AND = intersection; ids in ascending order.
§8	Control layer	<code>downloaded_books.txt</code> and <code>indexed_books.txt</code> ; an id is appended only after its step succeeded.
§9–10	Benchmarks	The 12 experiments; CSV <code>language,experiment,structure,dataset_size,repetition,metric,value,unit</code> ; 2 warm-ups + 5 measured runs; no network inside a measurement; the reference term counts.

Table 2: The common contract, `shared/SPEC.md`, in short.

1.3 What the group delivers

	Java	Python	C++
Complete pipeline: crawler, datalake, metadata, index, AND search, control	✓	✓	✓
Three datalake layouts and three persistent index structures	✓	✓	✓
Offline mode over <code>sample_dataset/raw/</code>	✓	✓	✓
The 12 SPEC experiments, results committed in the common CSV format	✓	✓	✓
Automated tests (all passing)	357	39	232

Table 3: Status of the three implementations on 4 October 2026.

2. System architecture

2.1 Pipeline overview

The data layer separates unstructured content from structured, queryable data:

- The **datalake** keeps every downloaded book as two plain-text files, its header and its body (the licence footer is dropped). It is the single source from which every datamart can be rebuilt.
- The **datamarts** hold what later stages query: the **metadata** of each book (title, author, language, release date and the paths of its files) in SQLite, and the **inverted index**, which maps each term to the sorted ids of the books that contain it.
- The **control layer** records which books are downloaded and which are indexed, and decides

the next operation.

- A **query module** answers AND queries over the inverted index, so the pipeline can be checked end to end (**search whale island** returns books 76, 84 and 2701 in the three languages).

Figure 1 follows one book through the data layer. The three implementations share these components and this data flow; they differ in how each component is written.

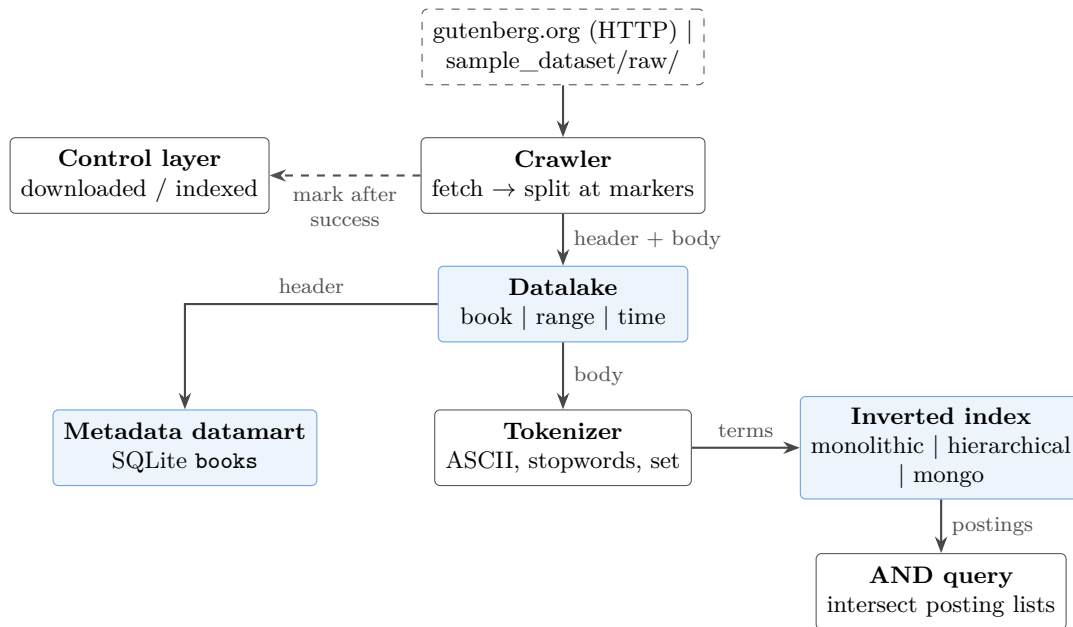


Figure 1: The data layer of Stage 1, common to the three implementations.

2.2 The structures under comparison

Structure	Layout	Expected trade-off
<i>Datalake</i>		
book	<ID>/header.txt, body.txt	Path computed from the id; one root entry per book.
range	01000-01999/<ID>.body.txt	Path computed from the id; at most 2,000 files per folder.
time	YYYYMMDD/HH/<ID>.body.txt	Arrival order and small folders; the path cannot be computed from the id.
<i>Inverted index</i>		
monolithic	One JSON {"term": [ids]}	Everything in memory: fast queries; each update rewrites the whole file.
hierarchical	inverted_index/ <L>/<term>.txt	Updates touch only a book's terms; one small file per term.
mongo	Collection inverted_index, unique index on term	Data in a server: cheap updates, shared by processes; a round trip per term.
<i>Metadata</i>		
sqlite	Table books + indexes on author, title	Fast lookups by author or title, slower inserts.
sqlite_no_index	The same table without those indexes	The baseline: full scans.

Table 4: The structures that the benchmarks compare.

2.3 Control layer

The control layer follows the decision process of the assignment, with two control files in `data/control/`: `downloaded_books.txt` and `indexed_books.txt`, one id per line.

1. **Index first.** If a book is downloaded but not indexed, it is indexed: its header is parsed into the metadata datamart and its body is tokenized and added to the inverted index.
2. **Otherwise download.** The next id of `shared/book_ids.txt` that is not in `downloaded_books.txt` is downloaded, split and stored in the datalake, so a book is never downloaded twice. Instead of the random ids of the assignment's example, the group uses a fixed list of 200 ids, so the three languages process exactly the same books and every id is known to have both markers.
3. **Update the state.** The id is appended to the matching control file only after the operation has been written to disk.

Java and C++ implement this loop literally: each `pipeline` step performs one download or one indexing, indexing first. Python processes each book completely in one step (download, then index into the three structures) and marks it after each phase; the effect on the control files is the same. In Stage 1 the control layer is sequential, as the assignment requires; parallel downloaders and indexers are left for later stages.

2.4 Reliability rules shared by the three implementations

- **Mark after doing.** An id is appended to a control file only after its step was written. The worst case after a crash is repeating one step, never losing a book.

- **Idempotent operations.** Saving a book, upserting metadata, adding a document to an index (sets, `$addToSet`) can be repeated safely after a failure.
- **A half-written control line is ignored.** If a process dies in the middle of an append, the fragment is not glued to the next id (“13” + “84” $\neq$ “1384”).
- **An ASCII-only tokenizer.** Unicode-aware lowercasing behaves differently in each language; walking the bytes against fixed ranges makes the three indexes identical.

3. How each language implements the data layer

3.1 Java

Stack. Java 17 (tested on OpenJDK 21) built with Maven; `java.net.http.HttpClient` for downloads, `sqlite-jdbc` for SQLite, Jackson for JSON, the synchronous MongoDB driver, and JUnit 5. About 4,800 lines in 44 classes and interfaces, plus 5,200 lines of tests.

Design. The implementation is object-oriented and organised around **interfaces**: `BookSource`, `Datalake`, `MetadataRepository` and `InvertedIndex`, each with several implementations. Every class receives its collaborators in the constructor, two factories map configuration names to classes, and a single assembly class, `SearchEngine`, connects the whole system, so the command line and the tests share exactly the same wiring.

- `AbstractFileDatalake` is a template method: the three layouts only decide *where* a book goes; the base class writes header then body atomically (temporary file plus `ATOMIC_MOVE`) and defines what counts as a complete book.
- `MonolithicJsonIndex` keeps a `TreeMap<String, TreeSet<Integer>` and rewrites the JSON when dirty; `HierarchicalFolderIndex` rewrites only changed term files; `MongoInvertedIndex` sends one unordered `bulkWrite` of upserts with `$addToSet`.
- `PipelineController.step()` performs at most one operation, indexing first, then downloading; `ControlFiles` appends only after success.
- The active datalake and index are chosen in `config.properties` or with `-D` options; `AppConfig` is the only class that builds paths, and fails at start-up on an unknown key or structure.

Testing. 357 tests at four levels: unit tests; *contract tests* that run the same cases against every implementation of `Datalake` (36) and `InvertedIndex` (33); integration tests with real SQLite and MongoDB; and end-to-end tests that run the whole program without network, kill it, reopen it, and check that nothing is downloaded twice. `SampleDatasetTest` checks the split byte by byte against `sample_dataset/book/`.

Benchmarks. Timed with `System.nanoTime`; memory read from the JVM heap after requesting a garbage collection. The two warm-ups also warm up the JIT compiler.

3.2 Python

Stack. Python 3.10+ (results with 3.12), `requests` and `pymongo`; everything else comes from the standard library (`sqlite3`, `json`, `pathlib`, `tracemalloc`). About 2,900 lines in 26 modules, the shortest of the three, plus 39 `pytest` tests.

Design. A modular design with abstract base classes (`abc.ABC`) for `Datalake` and `InvertedIndex`, frozen dataclasses with `__slots__` for the records, and plain functions for the stateless steps (`split_header_body`, `parse_metadata`, `tokenize`, `search`).

- **The pipeline writes every structure at once:** each book goes to the three datalakes and the three indexes in the same pass, and all of them are flushed before the book is marked as indexed. One run fills everything the benchmarks need, and the structures can never drift apart; the price is six writes per book.
- Atomic writes with a `.tmp` file and `os.replace`, in a single helper shared by the three layouts.
- `JavaRandom` and `java_shuffle` port `java.util.Random`, so the random lookup order and metadata queries are exactly Java's.
- A command line (`argparse`) with `pipeline`, `search` and `status`.

Testing. 39 unit tests, plus checks that run on the real data: every index benchmark calls `verify()` against an in-memory reference index, and `verify_results` checks the committed CSVs against the exact values of SPEC section 10. Its output was checked byte for byte against Java's on the 200 books.

Benchmarks. Timed with `time.perf_counter_ns`; memory measured with `tracemalloc` (objects allocated through Python's allocator). A first run on Windows 11 was many times slower, because NTFS and the antivirus slowed down the 129,000 small files of the hierarchical index, so the official results were taken on Linux.

3.3 C++

Stack. C++20 with CMake presets and Ninja (`make` and `make test` as shortcuts); SQLite3, libcurl and mongo-cxx-driver from the system; nlohmann/json and GoogleTest downloaded by CMake. About 5,200 lines in 97 header and source files, plus 3,500 lines of tests (232 tests).

Design. The logic lives in a static library, `stage1_core`, and the executable only parses arguments and dispatches. The code favours **free functions for decisions** (`split_book`, `extract_metadata`, `tokenize`, `query_and`, `next_control_action`), which are tested without disk or network, and small interfaces for what has side effects (`Datalake`, `IndexWriter`, `BookSource`, `HttpClient`, `Clock`).

- **RAII** owns every resource: the `sqlite3*` handle, prepared statements, the curl handle, temporary folders in the tests. `std::optional` marks expected absence (no markers, no book) and exceptions are kept for I/O errors.
- The index is an in-memory `std::unordered_map<std::string, std::set<int>>`, rebuilt at start-up; an `IndexWriter` persists it: `write` for the whole index, `update_terms` for the terms of one new book. MongoDB updates go in one unordered bulk write, as in Java.
- The `time` datalake remembers the paths it wrote, so its `locate` is a map lookup (Section 5.2 explains why this matters in the comparison).
- `JavaRandom`, a 48-bit linear congruential generator with Java's published algorithm, replaces `<random>`, whose output differs between `libc++` and `libstdc++`.

Testing. 232 GoogleTest tests with fake HTTP clients and clocks; the 11 that need MongoDB are skipped without a server. A development log (`cpp/docs/DEVLOG.md`, 72 entries) records each decision together with its measurements.

Benchmarks. Timed with `std::chrono::steady_clock`; memory read from the allocator (`mallinfo2` on Linux, `malloc_zone_statistics` on macOS).

3.4 The three implementations side by side

	Java	Python	C++
Paradigm	Object-oriented, interfaces, factories	Modules, ABCs, functions	Free functions + small interfaces, RAII
Production code	~4,800 lines, 44 classes	~2,900 lines, 26 modules	~5,200 lines, 97 files
Tests	357 (JUnit 5)	39 (pytest) + run-time checks	232 (GoogleTest)
External libraries	sqlite-jdbc, Jackson, Mongo driver	requests, pymongo	SQLite3, libcurl, mongo-cxx-driver, nlohmann/json
Build and install	Maven downloads everything	<code>pip install</code>	Compiler, CMake, Ninja; native libraries installed by hand
Structures in the pipeline	One of each, by configuration	All of them in the same run	One of each, by configuration
In-memory index	<code>TreeMap</code> of <code>TreeSet<Integer></code>	<code>dict</code> of <code>set</code>	<code>unordered_map</code> of <code>std::set<int></code>
Atomic file writes	Yes (temp + atomic move)	Yes (temp + <code>os.replace</code>)	No (relies on the control layer)
Memory metric	JVM heap after GC	<code>tracemalloc</code>	Allocator statistics

Table 5: The three implementations compared by construction.

3.5 Same output in the three languages

Before comparing speed, the group checked that the three programs compute the same thing. All of them produce:

- the same datalake: 400 files and 128,980,349 bytes for the 200 books, in every layout, and the 15 sample books byte-identical to `sample_dataset/book/`;
- the same index: 58,834 / 78,820 / 129,356 terms and 360,970 / 759,087 / 1,581,064 postings for $N = 50 / 100 / 200$, and a monolithic JSON of exactly 8,893,537 bytes;
- the same answer to the sample query “whale island”: books 76, 84 and 2701.

Every difference reported in the following sections is therefore a difference of *speed or memory*, never of results.

4. Design decisions

The assignment asks to justify the selected data structures and indexing strategies. Every structure is implemented in the three languages and benchmarked in Section 5; this section states the choice and the reasons, which the three implementations reached independently.

4.1 Datalake: book

- **Lookup.** The path of a book is computed from its id, so a lookup is one existence check: 2.5–3.3 μ s in C++ and Java, and the fastest layout in Python too. In the `time` layout the path depends on when the book was downloaded, so Java and Python must walk the hour folders (29 and 14 times slower), a cost that grows with every hour of ingestion.

- **Ingestion, incremental detection and recovery.** The three layouts write the same bytes at about the same speed, all of them detect exactly the 20 new books and recover the 20 interrupted ones with nothing lost or duplicated; **book** is the fastest or within a few milliseconds of it in every language.
- **Storage overhead.** 400 files in every layout; **book** adds one folder per book (200), a 1.5% block overhead, negligible at this scale.
- **Why not the recommended time layout.** Its strengths are traceability, small folders (20 entries) and the possibility of processing only the most recent folders. In Stage 1 the control files already say which books are new, so nothing benefits from that, while every lookup by id pays its cost. The layout stays implemented and selectable.
- **When to change.** With tens of thousands of books, **book** puts one entry per book in a single root folder; **range** keeps the direct lookup and bounds every folder.

4.2 Inverted index: monolithic, with mongo as the growth path

- **At the scale of Stage 1** (200 books, 129,356 terms) **monolithic** is the fastest to build, to query and, in Java and C++, to update, and the smallest on disk (8.9 MB), in the three languages.
- **Its limits are structural:** every update rewrites the whole file, and the whole index lives in memory (54–147 MB for 200 books depending on the language). Both grow linearly with the collection.
- **hierarchical** updates only the files of a book's terms and uses almost no memory, but its 129,356 small files take 530 MB of disk blocks for 7.3 MB of data (73 times).
- **mongo** keeps the index out of the process, supports concurrent readers and writers and updates one document per term. It is the structure to move to when the collection outgrows memory or when several processes share the index (Stage 2), at the cost of a network round trip per query term.
- **Indexing strategy, common to all structures.** Terms come from the shared ASCII tokenizer with stopwords removed; each book contributes its *set* of terms (no frequencies, no positions), and posting lists are kept sorted and without duplicates, so an AND query is a linear-time intersection that starts from the shortest list.

4.3 Metadata: SQLite with indexes on author and title

SQLite is embedded, needs no server and is part of the standard tooling of the three languages. With 100,000 rows, the two indexes make lookups by author 160–380 times faster than a full scan and keep them flat as the table grows, for a slower insertion that only matters during bulk loads. The comparison with other backends (PostgreSQL, MongoDB), optional in the assignment, was not carried out; Section 7.2 lists it as future work.

4.4 Shared implementation decisions

- **A fixed dataset of 200 real books**, listed in `shared/book_ids.txt`, instead of random ids: every language processes the same books, and the result can be checked against reference counts.
- **Atomic writes** (temporary file plus rename) in Java and Python, and **marking after doing** in the three, so a crash never leaves a half-written book counted as stored.

- **Configuration instead of code changes:** Java and C++ select the datalake and index in `config.properties`; Python keeps every structure up to date in each run.
- **MongoDB is optional:** the group's server runs from `docker-compose.yml`, and the three programs skip the `mongo` structure with a warning when it is not reachable.

Part	Choice	Because (holds in the three languages)
Datalake	<code>book</code>	Fastest lookup that survives a restart, fast change detection and recovery, same bytes as the others; the benchmarks also read it. <code>range</code> is the alternative for a far larger catalogue.
Inverted index	<code>monolithic</code>	Fastest build, queries and updates at 200 books, smallest on disk. Its update cost and memory grow with the index; beyond a few hundred to a few thousand books, <code>mongo</code> (or <code>hierarchical</code>) takes over.
Metadata	<code>sqlite</code> with indexes	Lookups by author and title stay flat as the table grows, for a modest insertion cost.

Table 6: Structures selected for the final implementation.

5. Benchmarks and results

This section presents the 12 experiments required by the assignment for the datalake, the metadata and the inverted index, and compares the three languages in each of them.

5.1 Methodology

5.1.1 Common rules

- **2 warm-ups and 5 measured runs** per experiment; this report uses the **median** of the five, which resists isolated pauses (garbage collection, disk flushes).
- **Preparation is not timed:** clearing folders and databases, opening connections and tokenizing the books happen before the clock starts.
- **No network inside a measurement:** the real books are read from a `book` datalake filled once by the pipeline.
- **Same data, same order:** a size N is the N books with the lowest ids; random choices (lookup order, metadata queries) come from the same `java.util.Random` sequence with seed 42 in the three languages; the `time` layout uses a simulated clock of 10 books per hour.
- **Verification before reporting:** every experiment checks its result (every book found, posting lists equal to an in-memory reference, 0 books lost) and stops instead of writing numbers if the check fails.

5.1.2 Experiments and data

Experiments	Structures	Data
<code>datalake_write, _lookup, _incremental, _recovery, _storage</code>	<code>book, range, time</code>	The 200 real books (129.0 MB)
<code>index_build, _query, _update, _memory, _disk</code>	<code>monolithic, hierarchical, mongo</code>	The 50, 100 and 200 real books with the lowest ids
<code>metadata_insert, _query</code>	<code>sqlite, sqlite_no_index</code>	1,000 / 10,000 / 100,000 generated rows (each author has 10 books, each title 2)

Table 7: The 12 experiments of SPEC sections 9–10.

The assignment’s *download and write throughput* is measured on the write side only: downloading depends on Project Gutenberg and the network, which would hide any difference between structures or languages, so the books are downloaded once by the pipeline and the benchmark times splitting-free storage of the 200 books. Metadata uses generated rows because 200 real books are too few to show how SQLite scales; the assignment asks for thousands to tens of thousands of rows.

5.1.3 Environment: what is directly comparable

Experiments	Java	Python	C++
Index (5 experiments)	Linux	Linux	Linux
Datalake (5 experiments)	Linux	Linux	macOS, Apple Silicon
Metadata (2 experiments)	Linux	Linux	macOS, Apple Silicon

Table 8: Where each set of results was measured. “Linux” is one machine: Ubuntu 24.04, 12 threads, 15 GB RAM, NVMe SSD, MongoDB 8.2.12 in Docker; OpenJDK 21 and Python 3.12.

Reading the comparison. The **index** experiments of the three languages, and every Java and Python experiment, ran on the same Linux machine: there, a difference is a difference of language and implementation. The C++ **datalake and metadata** results come from a different machine and operating system (APFS on Apple Silicon), so for those two families a gap with C++ mixes the language with the hardware; the figures mark those C++ bars with a hatched pattern. Memory is measured differently in each language (JVM heap, `tracemalloc`, allocator statistics), so its absolute values are only indicative; how it grows with N is comparable.

5.2 Datalake

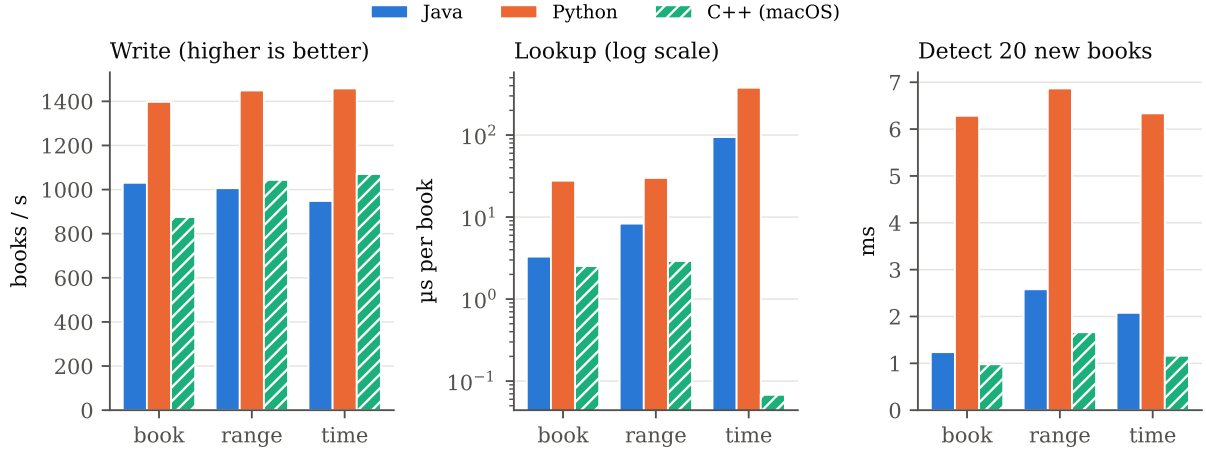


Figure 2: Datalake with the 200 real books, median of 5 runs. Hatched: C++, measured on macOS.

Metric	Layout	Java	Python	C++ [†]
Write (books/s)	book	1,031	1,398	876
	range	1,006	1,449	1,044
	time	948	1,458	1,071
Lookup (μ s/book)	book	3.3	27.8	2.5
	range	8.3	30.0	2.9
	time	94.5	377.9	0.07[‡]
Detect 20 new books (ms)	book	1.24	6.28	0.98
Recover 20 books (ms)	book	23.8	22.5	19.1

Table 9: Datalake timings, 200 books. [†] measured on macOS. [‡] an in-memory map of the paths written by the process, not a walk of the folders. Bold: the fastest language. All three recover the 20 damaged books with 0 lost and 0 duplicated, and store the same bytes (400 files, 129.0 MB).

- **Writing is bound by the file system, not the language.** The three languages write the same 129 MB in 400 files at 880–1,460 books/s, and the layouts differ by about 20% at most. Python is even the fastest: `write_text` hands the whole buffer to the operating system in one call, so the interpreter does very little per book.
- **Lookups show the cost of the language around each system call.** A book or range lookup is one path computation and one existence check. C++ and Java need 2.5–8 μ s; Python needs about 28 μ s, because building `Path` objects and calling the file system from interpreted code costs 8 times more than in Java on the same machine.
- **The time layout is a design, not a language, result.** Java and Python walk the hour folders, newest first, and are 29 and 14 times slower than their own `book` lookup. C++ remembers the paths it has written, which is fast but does not survive a restart, so its 0.07 μ s is not comparable with the other two.
- **Detecting new books** (listing the folders) is again 5 times slower in Python than in Java, while **recovery**, dominated by rewriting 20 books, is equal within 25% in the three.

Layout	Files	Folders	Fullest folder	Data (MB)	Allocated (MB)
book	400	200	200	129.0	130.9
range	400	47	220	129.0	130.3
time	400	21	20	129.0	130.2

Table 10: Storage overhead of the datalake, 200 books: identical in the three languages. Besides these files, every layout uses the same two control files.

Storage overhead (Table 10) does not depend on the language: the three write the same files. It shows the structural trade-off of each layout: **time** bounds every folder by the ingestion rate (20 entries), **book** puts one folder per book in the root, and **range** does not spread these ids well, because 110 of the 200 are below 1,000 and share one folder.

5.3 Inverted index

This is the family where the comparison is cleanest (same machine for the three languages) and where most of the program’s own work happens.

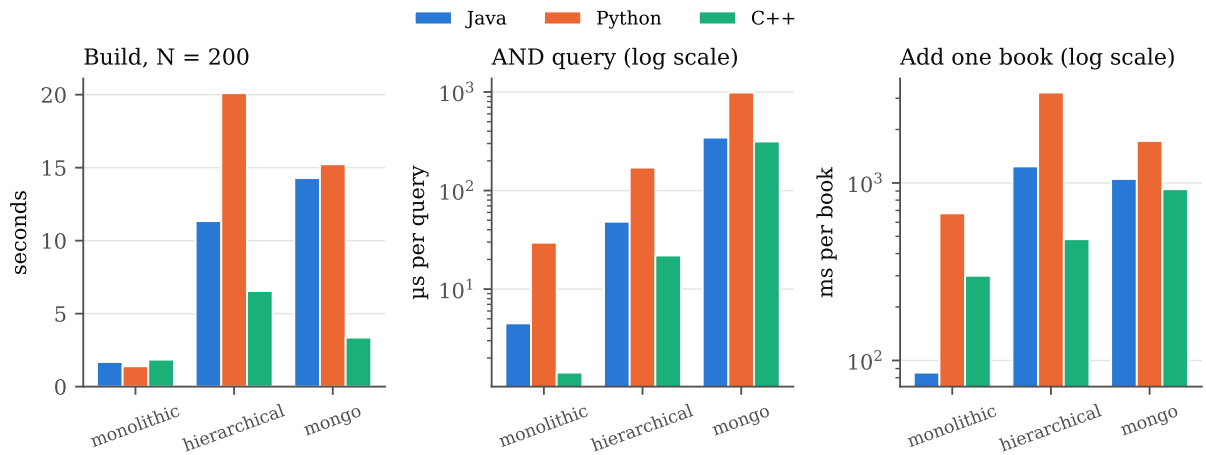


Figure 3: Inverted index with $N = 200$ books, median of 5 runs, same Linux machine for the three languages.

Metric ($N = 200$)	Structure	Java	Python	C++
Build (s)	monolithic	1.69	1.40	1.85
	hierarchical	11.34	20.10	6.55
	mongo	14.29	15.23	3.36
AND query (μ s)	monolithic	4.5	29.6	1.4
	hierarchical	48.3	171.1	22.0
	mongo	344.8	987.7	314.6
Add one book (ms)	monolithic	85.4	674.0	300.0
	hierarchical	1,239	3,230	482
	mongo	1,053	1,723	922

Table 11: Inverted index, $N = 200$: 129,356 terms and 1,581,064 postings in every case.

5.3.1 Queries: the clearest language effect

An AND query tokenizes a few words, fetches their posting lists and intersects them. On the **monolithic** index everything is in memory, so the time is pure program execution: C++ answers in $1.4\ \mu\text{s}$, Java in $4.5\ \mu\text{s}$ (3.2 times) and Python in $29.6\ \mu\text{s}$ (21 times). Python sorts each posting list out of a **set** and intersects it in an interpreted loop; Java iterates boxed **Integers** from a **TreeSet**; C++ walks contiguous integers.

When the query has to leave the process, the language matters less: on **hierarchical** (one file read per term) the spread shrinks to 2.2 times for Java and 7.8 times for Python, and on **mongo** (one round trip to the server per term) Java is only 10% slower than C++. The slower the storage, the smaller the share of the time that the language controls.

5.3.2 Build and update: language and design mixed

- **monolithic build**: the three are within 33% of each other, and Python is the fastest, because filling dictionaries and `json.dumps` run in C inside the interpreter.
- **hierarchical build** creates 129,356 files: C++ needs 6.5 s, Java 11.3 s (1.7 times) and Python 20.1 s (3.1 times), the overhead of each language per file operation.
- **mongo build** is 4.3 times faster in C++ for a design reason: it fills an empty collection with `insert_many` of whole documents, while Java and Python send one upsert with `$addToSet` per term. When the three use the same upsert (the update experiment), C++ and Java are within 15%.
- **monolithic update** rewrites the whole 8.9 MB JSON after each book. Here **Java is the fastest** (85 ms), ahead of C++ (300 ms) and Python (674 ms): Jackson serialises the sorted map directly, the C++ writer goes through nlohmann/json, and Python sorts the ids of all 129,356 terms before every write. With the **monolithic** build, it is one of the two index experiments where C++ is not the fastest, and it is the write path of the structure the group selected.

5.3.3 Scaling with the size of the collection

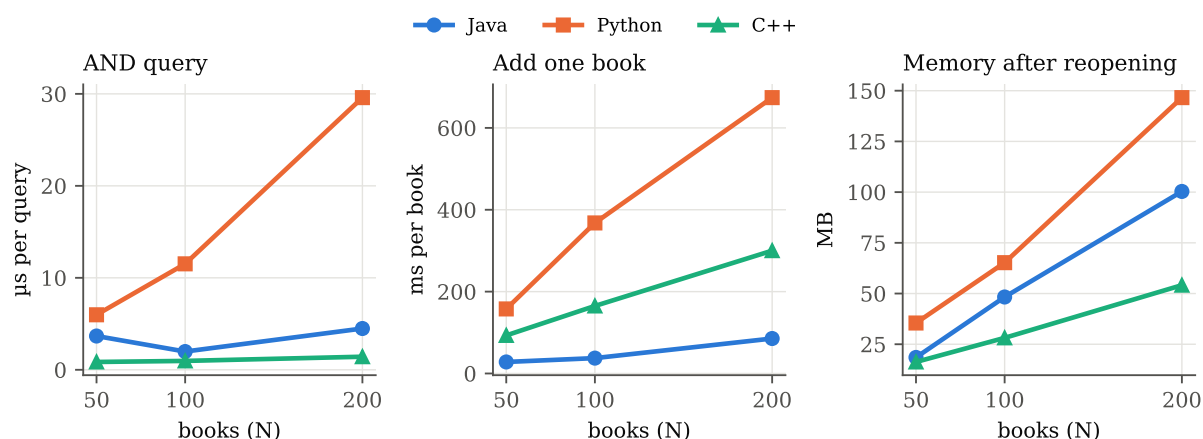


Figure 4: The **monolithic** index as the collection grows from 50 to 200 books.

Figure 4 shows how the structure chosen by the three languages scales. The ordering of the languages does not change with N , but the slopes do. Python's update cost grows fastest (158

→ 674 ms, 4.3 times for 4 times the books) because its per-flush sort grows with the index; C++ grows 3.2 times (93 → 300 ms) and Java 3.0 times (28 → 85 ms). Query times stay in the microsecond range in the three languages.

5.4 Memory and disk

Memory after reopening the <code>monolithic</code> index (MB)	Java	Python	C++
$N = 50$	18.5	35.5	16.3
$N = 100$	48.3	65.2	28.1
$N = 200$	100.3	146.6	54.1

Table 12: Memory held by the open `monolithic` index; `hierarchical` and `mongo` keep almost nothing (<50 KB) in the three languages.

The same 8.9 MB of JSON occupies 54 MB as C++ `std::set<int>`, 100 MB as Java `TreeSets` of boxed `Integers` (about 63 bytes per posting) and 147 MB as Python `sets` of `int` objects. The representation of a posting, not the data, sets the memory cost, and the gap between languages widens as the index grows, reaching 1 : 1.9 : 2.7 (C++ : Java : Python) at $N = 200$. On disk the three languages write the same bytes (Table 13).

Structure ($N = 200$)	Data (MB)	Files	Allocated (MB)	Allocated / data
<code>monolithic</code>	8.9	1	8.9	1.0
<code>hierarchical</code>	7.3	129,356	530.0	73.1
<code>mongo</code>	13.1–14.1	—	—	—

Table 13: Index on disk. The file-based structures are byte-identical in the three languages; for `mongo` the size is the collection plus its index, as reported by the server.

Every term file of `hierarchical` takes at least one 4 KB block, so 7.3 MB of postings reserve 530 MB: the “very large number of small files” that the assignment warns about, measured.

5.5 Metadata (SQLite)

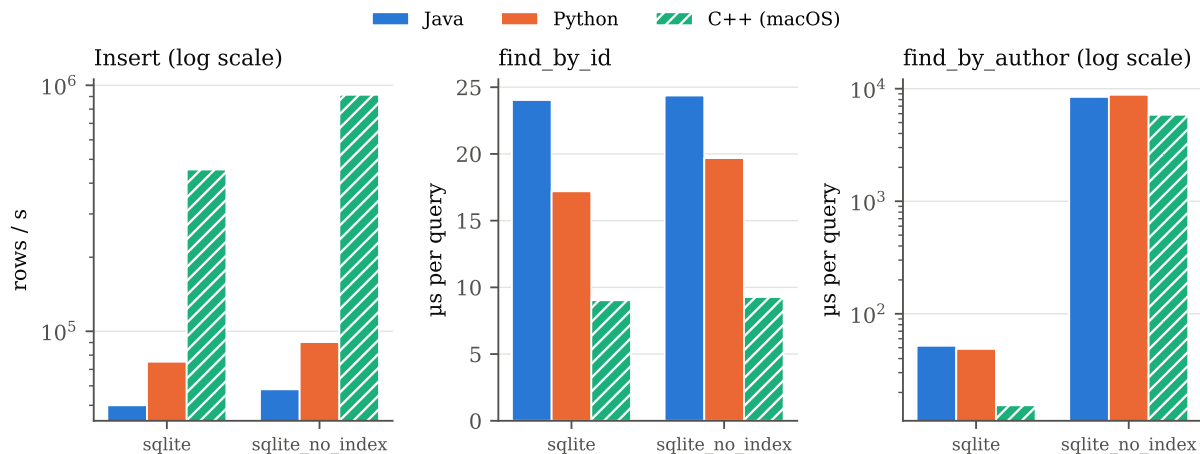


Figure 5: Metadata with 100,000 generated rows, median of 5 runs. Hatched: C++, measured on macOS.

Metric ($N = 100,000$)	Variant	Java	Python	C++ [†]
Insert (rows/s)	sqlite	50,036	75,256	454,570
	sqlite_no_index	58,103	90,399	915,179
find_by_id (μ s)	sqlite	24.0	17.2	9.0
find_by_author (μ s)	sqlite	52.0	48.8	15.4
	sqlite_no_index	8,460	8,834	5,869

Table 14: Metadata. [†] measured on macOS.

- **The conclusion about the structure is the same in the three languages:** without the `author` index a lookup is a full scan that grows with the table, 160–380 times slower at 100,000 rows; with it, lookups stay flat.
- **Between Java and Python (same machine), Python is faster here:** SQLite is C code in both, and `executemany` in the `sqlite3` module hands a whole batch to C code, while JDBC adds the cost of its binding layer to every row. Python inserts 1.5 times faster; queries by author are equal within 10%.
- **C++ calls the SQLite C API directly**, with no binding layer, and is 3–9 times faster. Part of this gap is the different machine and disk, so it cannot be read as a pure language result; the `sqlite_no_index` scans, which are pure SQLite work, show the machine effect alone at about 1.5 times.

5.6 Overall view

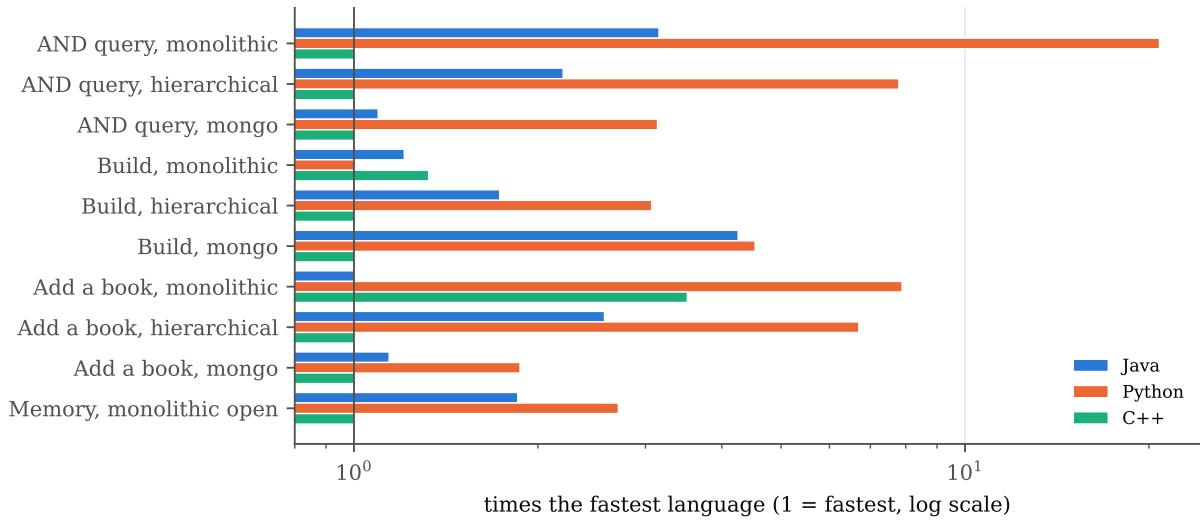


Figure 6: Cost of each language relative to the fastest one (= 1) in the index experiments, $N = 200$, all measured on the same Linux machine.

Where the time goes	What the measurements show	Ranking
In the program's own loops (queries, intersections, per-term work)	C++ fastest; Java 1.1–3.2 times; Python 3–21 times	C++ > Java ≫ Python
In the operating system (writing large files)	Within 20%; Python slightly ahead	All equal
Many small system calls (lookups, 129,356 files)	Interpreter overhead per call: Python 3–8 times Java	C++ > Java > Python
In C libraries (JSON, SQLite)	Depends on the binding: Python's C modules beat JDBC; Jackson beats nlohmann/json	Mixed
In an external server (MongoDB)	Round trips dominate; Java within 15% of C++	C++ ≈ Java > Python
Memory of in-memory structures	1 : 1.9 : 2.7 (C++ : Java : Python)	C++ > Java > Python

Table 15: Where the differences between languages come from.

Figure 6 and Table 15 summarise the comparison. **C++** is the fastest in 7 of the 9 time-based index experiments and uses the least memory. **Java** wins the **monolithic** update and is the second fastest in the other eight; its gap with C++ is at most 3.2 times on queries and 4.3 times overall, the latter in the **mongo** build, which is a design difference. **Python** is competitive only where the work is delegated to C code or to the operating system; in its own loops it is the slowest by a wide margin.

6. Choosing the language for the project

6.1 What the project needs

The language chosen now will carry the search engine into the next stages, where the data layer grows (more books, continuous ingestion) and is used by services (indexing and query processes running at the same time, likely distributed). From Stage 1 the requirements are:

1. **Low query latency**, since queries are what users wait for.
2. **Indexing and ingestion throughput**, to keep up with a growing collection.
3. **Reasonable memory per posting**, while the index lives in memory.
4. **Development effort and maintainability** for a team of three.
5. **Portability and deployment** on the members' machines, the grader's machine and containers.
6. **Ecosystem for the next stages**: database drivers, HTTP and service frameworks, distributed processing.
7. **Safety of a shared codebase**: catching errors at compile time, no memory corruption.

6.2 How each language meets them

Criterion	Weight	Java	Python	C++
Query latency	20	2 — 1.1–3.2× C++	1 — 3–21× C++	3 — fastest everywhere
Indexing throughput	20	2 — best monolithic update, 2nd elsewhere	1 — slowest except monolithic build	3 — fastest in 4 of 6
Memory	10	2 — 1.9× C++	1 — 2.7× C++	3 — least
Development effort	15	2 — 4,800 lines, verbose but plain	3 — 2,900 lines, the shortest	1 — 5,200 lines, manual resource handling
Portability, deployment	10	3 — JDK + Maven fetch everything	3 — <code>pip install</code>	1 — toolchain and native drivers per platform
Ecosystem for next stages	15	3 — JVM big-data and service stack	2 — strong for data analysis	1 — fewest ready-made pieces
Safety of shared code	10	3 — static types, memory safe	1 — dynamic types	2 — static types, not memory safe
Weighted score (of 3)	100	2.35	1.65	2.10

Table 16: Decision matrix. Scores: 3 best, 1 worst, from the measurements of Section 5.2 and the experience of building the three implementations.

Some notes on the scores:

- **Performance (criteria 1–3)** comes from the index experiments, the only ones measured on the same machine for the three languages.
- **Deployment** was a real issue during Stage 1: the C++ module needs a C++20 compiler, CMake, Ninja, SQLite, libcurl and mongo-cxx-driver 4.x, which on Linux has to be built from its sources, while Java needs only a JDK and Maven, and Python only `pip`. A C++ test also failed only on Linux and had to be reproduced in a container.

- **Ecosystem.** The official MongoDB, SQLite and HTTP clients exist for the three, but the tools typically used to scale a data system across machines (distributed caches, message queues, Hadoop and Spark, mature web frameworks) are native to the JVM, available from Python, and rarely first-class in C++.
- **Sensitivity.** With the three performance criteria weighted at 50%, Java wins. C++ only overtakes Java if performance weighs more than 60% of the decision. Python scores at least as low as Java on every performance criterion and does not beat it on the rest, so no weighting makes it the first choice for the core system.

6.3 Recommendation

The group recommends Java as the main language of the project. It is the best balance between the measured performance and the needs of a system that will grow and be developed by a team: within 1.1–3.2 times of C++ on every query, second only to C++ in the other index experiments, and the fastest at updating the selected `monolithic` index; 2–21 times faster than Python where the program’s own code runs; the simplest to deploy and to test (357 tests, contract tests over every structure); and the language with the strongest ecosystem for the distributed stages to come.

The other two implementations keep a role:

- **C++** is the choice if a later stage needs a single, latency-critical query service on a memory-constrained node: it answers `monolithic` queries 3.2 times faster than Java and holds the index in half the memory. Its cost is development time, portability and the lack of atomic writes in the current module.
- **Python** is the choice for everything around the system: analysis of results, the report charts, quick prototypes and data exploration. Its benchmarks show it is competitive when the heavy work runs in C (SQLite, JSON) or in the operating system, but its interpreted per-term loops make it the wrong choice for the indexing and query core.

7. Conclusions and future improvements

7.1 Conclusions

- The group delivered the Stage 1 data layer three times, in Java, Python and C++, with the same contract, the same outputs byte for byte, and the same 12 benchmark experiments in a common CSV format.
- The structures to use do not depend on the language: `book`, `monolithic` and SQLite with indexes win in the three, and the three show the same limits (`monolithic`’s growing update cost and memory, `hierarchical`’s 73-fold block overhead, `time`’s slow lookups).
- The language matters in proportion to how much of the time is spent in the program’s own code: up to 21 times for in-memory queries, about 1.1 times when a server or the file system does the work. C++ is the fastest, Java close behind, Python several times slower in its own loops.
- Some of the largest gaps come from design rather than language (C++’s `insert_many` build and in-memory `time` lookup; the serialiser behind each `monolithic` update). Comparing implementations of the same contract makes these visible.
- Weighing performance with development, deployment, safety and ecosystem, **Java** is the

language that best fits the project.

7.2 Limitations and future improvements

- **One machine for everything.** The C++ datalake and metadata experiments should be rerun on the Linux machine used for the rest, to remove the hardware factor from those comparisons.
- **Larger collections.** At 200 books the `monolithic` index still wins; the point at which another structure takes over (extrapolated at roughly 700 books in C++, where `hierarchical` becomes cheaper to update, and 2,000–2,500 in Java, where `mongo` does) depends on the language and should be measured with thousands of books.
- **Aligning the designs that are not language-bound:** the same Mongo build strategy, a `time` lookup that survives a restart, and grouped flushes in Python, so that the remaining differences are only the languages’.
- **Other metadata backends.** The optional comparison of SQLite with a client–server relational database (PostgreSQL) and a NoSQL store (MongoDB) would show the cost of concurrent access, which Stage 2 will need.
- **Concurrency.** Stage 1 is single-threaded in the three languages; the next stages will need parallel tokenization and concurrent queries, where the languages differ again (threads in Java and C++, processes in Python because of the global interpreter lock).

A. Repository and reproduction

A.1 Repository

The source code is hosted at https://github.com/BigData-Ulpgc/stage_1, following the `https://github.com/<group_name>/stage_1` format. It contains:

- a `README.md` with the architecture, the requirements of each language, and the commands to build, test, run the pipeline and run the benchmarks of each implementation;
- `shared/`: the common contract (`SPEC.md`), the 200 book ids, the stopwords and the query workload;
- `sample_dataset/`: the first 15 books, raw (`raw/pg<ID>.txt`, the pipeline input) and already split (`book/<ID>/`, the expected output), so instructors can test the pipeline without network;
- `java/stage1/`, `python/stage_1/` and `cpp/`: the three implementations, each with its own guide, technical report and committed benchmark results;
- `docs/`: this report and its sources;
- a Git history of more than 150 commits between 17 September and 4 October 2026 that shows how the three implementations progressed.

A.2 Quick test with the sample dataset

Without network, each implementation runs the pipeline on the 15 sample books and answers the same query with books 76, 84 and 2701:

```
java -cp "$CP" es.ulpgc.bigdata.Main pipeline 30 --offline          # java/stage1/
python -m src.main.bigdata.main pipeline 15 --offline-source ../../sample_dataset/raw  #
python/stage_1/
```

```
./build/release/search_engine_stage1 pipeline 30 --offline      # cpp/
... search whale island
```

A.3 Reproducing the benchmarks

From the repository root, `docker compose up -d` starts the group's MongoDB (optional: without it, the mongo structure is skipped with a warning).

Java (from `java/stage1/`):

```
mvn package
mvn -q dependency:build-classpath -Dmdep.outputFile=target/classpath.txt
CP="target/classes:$(cat target/classpath.txt)"
java -cp "$CP" es.ulpgc.bigdata.Main pipeline 400
java -cp "$CP" es.ulpgc.bigdata.benchmark.DatalakeBenchmark data/datalake/book
java -cp "$CP" es.ulpgc.bigdata.benchmark.IndexBenchmark 50,100,200 data/datalake/book
java -cp "$CP" es.ulpgc.bigdata.benchmark.MetadataBenchmark 1000,10000,100000
```

Python (from `python/stage_1/`):

```
pip install -r ../requirements.txt pytest
python -m src.main.bigdata.main pipeline 200
python -m src.main.bigdata.benchmark.benchmark_runner data/datalake/book
python -m src.main.bigdata.benchmark.benchmark_runner
python -m src.main.bigdata.benchmark.verify_results
```

C++ (from `cpp/`):

```
make && make test
B=./build/release/search_engine_stage1
$B pipeline 400
$B benchmark datalake_write      # one call per experiment, 12 in total
$B benchmark index_build
$B benchmark metadata_query
```

This report (from `docs/report/`): `uv run -with matplotlib python charts.py` reads the committed CSVs of the three languages and draws the figures; `pdflatex report.tex` twice builds the PDF.